

Registration number	2025-BSCPE-118
Name	Muhammad Zain

MINI Project

University Admission Management System

Objective:

Design and implement a console-based University Admission Management System using 3 Tier Architecture:

- Presentation Layer (Console Interaction)
- Business Logic Layer (BLL) • Data Layer (DL)

Task

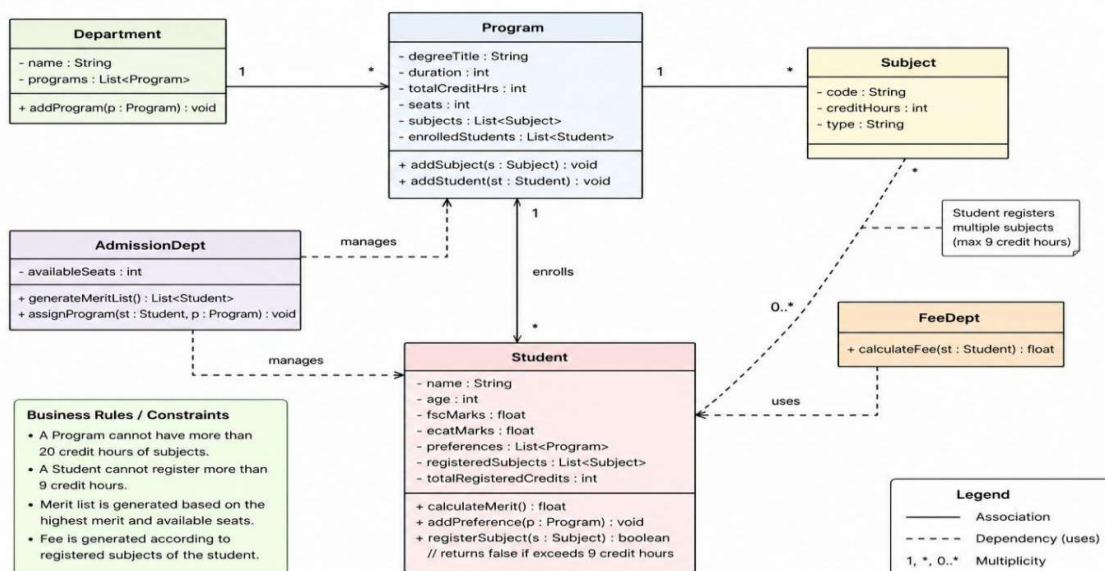
Academic branch offers different programs within different departments. Each program has a degree title and duration of degree.

Students apply for admission in the university and provide their name, age, FSC, and ECAT marks, and select any number of preferences among the available programs.

The admission department prepares a merit list according to the highest merit and available seats and registers selected students in the program.

The academic branch also adds subjects for each program. A subject has a subject code, credit hours, and subject type. A program cannot have more than 20 credit hours for subjects. A student registers multiple subjects but cannot take more than 9 credit hours.

The fee department generates fees according to the registered subjects of the students.



1. Presentation Layer (Console Layer)

Handles input/output via console

Classes

- Menu
- StudentView
- ProgramView
- AdmissionView

Responsibilities

- Display menus
- Take user input (student data, preferences)
- Show:
 - Merit list
 - Program allocation
 - Registered subjects
 - Fee details
 - Call functions from Business Logic Layer

Example Tasks

- Show menu: Add Student / View Merit / Register Subjects
- Print results on screen

Code:

AdmissionView

```
using System;
using System.Collections.Generic;
namespace UniversityAdmissionSystem
{
    public class AdmissionView
    {
        public static void GenerateMeritList()
        {
            List<Student> studentsWithPrefs = Student.GetWithPreferences();
            if (studentsWithPrefs.Count == 0)
            {
                Console.WriteLine("\nNo students with program preferences found.");
                Console.WriteLine("Add students and their program preferences first.");
                return;
            }
            MeritCalculator.DisplayMeritList();
        }
    }
}
```

```

        public static void ProcessAdmission()
        {
            List<Student> studentsWithPrefs = Student.GetWithPreferences();
            if (studentsWithPrefs.Count == 0)
            {
                Console.WriteLine("\nNo students with program preferences found.");
                return;
            }

            List<Program> availablePrograms = Program.GetAvailable();
            if (availablePrograms.Count == 0)
            {
                Console.WriteLine("\nNo programs with available seats found.");
                return;
            }

            Console.WriteLine("\nProcessing admission based on merit and preferences...");
            string result = AdmissionManager.ProcessAdmission();
            Console.WriteLine(result);
        }

        public static void ViewAdmittedStudents()
        {
            AdmissionManager.DisplayAdmittedStudents();
        }

        public static void ViewAllocationDetails()
        {
            AdmissionManager.DisplayAllocationDetails();
        }

        private static void DisplayAdmittedStudentList()
        {
            List<Student> admitted = Student.GetAdmitted();
            if (admitted.Count == 0) return;

            Console.WriteLine("\nAdmitted Students:");
            Console.WriteLine("ID          | Name          | Program");
            Console.WriteLine("-----|-----|-----");
            Console.WriteLine("-----|-----");
            foreach (Student s in admitted)
            {
                string programName = s.AllocatedProgram != null ? s.AllocatedProgram.DegreeTitle : "N/A";
                Console.WriteLine($"{s.StudentId,-6} | {s.Name,-13} | {programName,-24} | {s.CalculateMerit(),5:F2}");
            }
            Console.WriteLine();
        }

        public static void RegisterSubjects()
        {
            try
            {

```

```
Console.WriteLine("\n===== SUBJECT REGISTRATION =====\n");
Console.WriteLine(" b. Back to Main Menu\n");
DisplayAdmittedStudentList();

string studentId = InputValidator.GetValidString("Enter Student ID: ", false, true);
Student student = Student.GetById(studentId);

if (student == null)
{
    Console.WriteLine("Student not found.");
    return;
}

if (!student.IsAdmitted)
{
    Console.WriteLine("This student is not admitted to any program.");
    Console.WriteLine("Process admission first.");
    return;
}

if (student.AllocatedProgram == null || student.AllocatedProgram.Subjects.Count == 0)
{
    Console.WriteLine("No subjects available for this program.");
    return;
}

Console.WriteLine($"Available subjects for {student.AllocatedProgram.DegreeTitle}:");
Console.WriteLine("Code | Subject Name | Credit Hours | Type");
Console.WriteLine("-----|-----|-----");
foreach (Subject s in student.AllocatedProgram.Subjects)
{
    Console.WriteLine($"{s.SubjectCode,-6} | {s.Name,-19} | {s.CreditHours,12} | {s.SubjectType}");
}

Console.WriteLine($"Max credit hours allowed: {student.GetMaxCreditHours()}");

string input = InputValidator.GetValidString("Enter subject codes separated by commas (e.g., CS101,MATH101):\n", false, true);
List<string> subjectCodes = new List<string>();

foreach (string code in input.Split(','))
{
    string trimmed = code.Trim();
    if (!string.IsNullOrEmpty(trimmed))
        subjectCodes.Add(trimmed);
}

string result = RegistrationManager.RegisterSubjects(studentId, subjectCodes);
```

```
        Console.WriteLine("\n" + result);

        if (result.Contains("Successfully registered"))
        {
            RegistrationManager.DisplayRegisteredSubjects(studentId);
        }
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void ViewRegisteredSubjects()
{
    try
    {
        Console.WriteLine("\n===== VIEW REGISTERED SUBJECTS =====");
        Console.WriteLine("    b. Back to Main Menu\n");
        DisplayAdmittedStudentList();

        string studentId = InputValidator.GetValidString("Enter Student ID: ", false, true);
        Student student = Student.GetById(studentId);

        if (student == null)
        {
            Console.WriteLine("Student not found.");
            return;
        }

        if (student.RegisteredSubjects.Count == 0)
        {
            Console.WriteLine("\nNo subjects registered for this student.");
            return;
        }

        RegistrationManager.DisplayRegisteredSubjects(studentId);
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void ViewFee()
{
    try
    {
        Console.WriteLine("\n===== VIEW FEE DETAILS =====");
        Console.WriteLine("    b. Back to Main Menu\n");
        DisplayAdmittedStudentList();

        string studentId = InputValidator.GetValidString("Enter Student ID: ", false, true);
    }
}
```

```
        Student student = Student.GetById(studentId);

        if (student == null)
        {
            Console.WriteLine("Student not found.");
            return;
        }

        if (student.RegisteredSubjects.Count == 0)
        {
            Console.WriteLine("\nNo subjects registered. Please register subjects first.");
            return;
        }

        FeeCalculator.DisplayFeeDetails(studentId);
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void ViewAllFees()
{
    FeeCalculator.DisplayAllAdmittedStudentsFee();
}
}
```

Menu

```
using System;
using System.Collections.Generic;

namespace UniversityAdmissionSystem
{
    public class NavigationBackException : Exception { }

    public class InputValidator
    {
        public static string GetValidString(string prompt)
        {
            return GetValidString(prompt, false, false);
        }

        public static string GetValidString(string prompt, bool allowEmpty)
        {
            return GetValidString(prompt, allowEmpty, false);
        }

        public static string GetValidString(string prompt, bool allowEmpty, bool allowBack)
        {
            string input;
            do
            {
                Console.Write(prompt);
                input = Console.ReadLine();
            }
            while (input.Length == 0 && allowEmpty || input.Contains("\b") && allowBack);
        }
    }
}
```

```
        if (allowBack && IsBack(input))
            throw new NavigationBackException();
        if (string.IsNullOrEmpty(input))
        {
            if (allowEmpty)
                return "";
            Console.WriteLine("Error: This field cannot be empty.");
        }
    } while (string.IsNullOrEmpty(input));
    return input.Trim();
}

public static int GetValidInteger(string prompt, int min, int max)
{
    return GetValidInteger(prompt, min, max, null, false);
}

public static int GetValidInteger(string prompt, int min, int max, in
t? currentValue)
{
    return GetValidInteger(prompt, min, max, currentValue, false);
}

public static int GetValidInteger(string prompt, int min, int max, in
t? currentValue, bool allowBack)
{
    int value;
    while (true)
    {
        Console.Write(prompt);
        string input = Console.ReadLine();
        if (allowBack && IsBack(input))
            throw new NavigationBackException();
        if (string.IsNullOrEmpty(input) && currentValue.HasValue
)
            return currentValue.Value;
        if (int.TryParse(input, out value))
        {
            if (value >= min && value <= max)
                return value;
            Console.WriteLine($"Error: Please enter a number between
{min} and {max}.");
        }
        else
        {
            Console.WriteLine("Error: Please enter a valid integer.")
;
        }
    }
}

public static double GetValidDouble(string prompt, double min, double
max)
{
    return GetValidDouble(prompt, min, max, null, false);
}

public static double GetValidDouble(string prompt, double min, double
max, double? currentValue)
{

```

```
        return GetValidDouble(prompt, min, max, currentValue, false);
    }

    public static double GetValidDouble(string prompt, double min, double
max, double? currentValue, bool allowBack)
    {
        double value;
        while (true)
        {
            Console.Write(prompt);
            string input = Console.ReadLine();
            if (allowBack && IsBack(input))
                throw new NavigationBackException();
            if (string.IsNullOrEmpty(input) && currentValue.HasValue
)
                return currentValue.Value;
            if (double.TryParse(input, out value))
            {
                if (value >= min && value <= max)
                    return value;
                Console.WriteLine($"Error: Please enter a number between
{min} and {max}.");
            }
            else
            {
                Console.WriteLine("Error: Please enter a valid number.");
            }
        }
    }

    public static string GetValidChoice(string prompt, string[] validOpti
ons, bool allowBack)
    {
        while (true)
        {
            Console.Write(prompt);
            string input = Console.ReadLine();
            if (allowBack && IsBack(input))
                throw new NavigationBackException();
            if (string.IsNullOrEmpty(input))
            {
                Console.WriteLine("Error: This field cannot be empty.");
                continue;
            }
            string trimmed = input.Trim();
            foreach (string option in validOptions)
            {
                if (trimmed.Equals(option, StringComparison.OrdinalIgnoreCase
Case))
                    return option;
            }
            Console.WriteLine($"Error: Please enter one of the following:
{string.Join(", ", validOptions)}.");
        }
    }

    public static bool GetYesNo(string prompt)
    {
        return GetYesNo(prompt, false);
    }
}
```

```
public static bool GetYesNo(string prompt, bool allowBack)
{
    while (true)
    {
        Console.Write(prompt);
        string input = Console.ReadLine().ToLower();
        if (allowBack && IsBack(input))
            throw new NavigationBackException();
        if (input == "y" || input == "yes")
            return true;
        if (input == "n" || input == "no")
            return false;
        Console.WriteLine("Please enter y or n.");
    }
}

private static bool IsBack(string input)
{
    string trimmed = input?.Trim().ToLower();
    return trimmed == "b" || trimmed == "back";
}

public class Menu
{
    public static void ShowMainMenu()
    {
        while (true)
        {
            try { Console.Clear(); } catch { }
            Console.WriteLine("=====
=====");
            Console.WriteLine("  UNIVERSITY ADMISSION MANAGEMENT SYSTEM
");
            Console.WriteLine("=====
=====\\n");

            ShowDynamicMenu();

            Console.Write("\\nEnter your choice: ");
            string choice = Console.ReadLine();

            if (string.IsNullOrEmpty(choice) || !int.TryParse(choice, out
_))
            {
                Console.WriteLine("\\nInvalid choice. Please enter a valid
number.");
                Console.WriteLine("\\nPress Enter to continue...");
                Console.ReadLine();
                continue;
            }

            ProcessChoice(choice);
        }
    }

    public static void ShowDynamicMenu()
    {
        bool hasStudents = Student.Any();
```

```
bool hasPrograms = Program.Any();
bool hasSubjects = Subject.Any();
bool hasStudentsWithPrefs = Student.AnyWithPreferences();
bool hasAvailablePrograms = Program.AnyAvailable();
bool hasAdmittedStudents = Student.AnyAdmitted();

Console.WriteLine("STUDENT MANAGEMENT:");
Console.WriteLine(" 1. Add Student");
if (hasStudents)
{
    Console.WriteLine(" 2. View All Students");
    Console.WriteLine(" 3. Edit Student");
    Console.WriteLine(" 4. Delete Student");
    Console.WriteLine(" 5. Add Program Preferences");
}

Console.WriteLine("\nPROGRAM MANAGEMENT:");
Console.WriteLine(" 6. Add Program");
if (hasPrograms)
{
    Console.WriteLine(" 7. View All Programs");
    Console.WriteLine(" 8. Edit Program");
    Console.WriteLine(" 9. Delete Program");
}
Console.WriteLine("10. Add Subject");
if (hasSubjects)
{
    Console.WriteLine("11. View All Subjects");
    Console.WriteLine("12. Edit Subject");
    Console.WriteLine("13. Delete Subject");
}
if (hasPrograms && hasSubjects)
{
    Console.WriteLine("14. Add Subject to Program");
}

Console.WriteLine("\nADMISSION PROCESS:");
if (hasStudentsWithPrefs)
{
    Console.WriteLine("15. Generate Merit List");
    if (hasAvailablePrograms)
        Console.WriteLine("16. Process Admission");
}

if (hasAdmittedStudents)
{
    Console.WriteLine("17. View Admitted Students");
    Console.WriteLine("18. View Allocation Details");
    Console.WriteLine("19. Register Subjects");

    if (Student.AnyRegisteredSubjects())
    {
        Console.WriteLine("20. View Registered Subjects");
        Console.WriteLine("21. View Fee Details");
        Console.WriteLine("22. View All Fees");
    }
}

Console.WriteLine("\n 0. Exit");
}
```

```
public static void ProcessChoice(string choice)
{
    bool hasStudents = Student.Any();
    bool hasPrograms = Program.Any();
    bool hasSubjects = Subject.Any();
    bool hasStudentsWithPrefs = Student.AnyWithPreferences();
    bool hasAvailablePrograms = Program.AnyAvailable();
    bool hasAdmittedStudents = Student.AnyAdmitted();
    bool anyRegisteredSubjects = Student.AnyRegisteredSubjects();

    switch (choice)
    {
        case "1":
            StudentView.AddStudent();
            break;
        case "2":
            if (!hasStudents) Console.WriteLine("\nNo students to display.");
            else StudentView.ViewAllStudents();
            break;
        case "3":
            if (!hasStudents) Console.WriteLine("\nNo students to edit.");
            else StudentView.EditStudent();
            break;
        case "4":
            if (!hasStudents) Console.WriteLine("\nNo students to delete.");
            else StudentView.DeleteStudent();
            break;
        case "5":
            if (!hasStudents) Console.WriteLine("\nNo students found. Add a student first.");
            else StudentView.AddProgramPreferences();
            break;
        case "6":
            ProgramView.AddProgram();
            break;
        case "7":
            if (!hasPrograms) Console.WriteLine("\nNo programs to display.");
            else ProgramView.ViewAllPrograms();
            break;
        case "8":
            if (!hasPrograms) Console.WriteLine("\nNo programs to edit.");
            else ProgramView.EditProgram();
            break;
        case "9":
            if (!hasPrograms) Console.WriteLine("\nNo programs to delete.");
            else ProgramView.DeleteProgram();
            break;
        case "10":
            ProgramView.AddSubject();
            break;
        case "11":
            if (!hasSubjects) Console.WriteLine("\nNo subjects to display.");
    }
}
```

```
        else ProgramView.ViewAllSubjects();
        break;
    case "12":
        if (!hasSubjects) Console.WriteLine("\nNo subjects to edi
t.");
        else ProgramView.EditSubject();
        break;
    case "13":
        if (!hasSubjects) Console.WriteLine("\nNo subjects to del
ete.");
        else ProgramView.DeleteSubject();
        break;
    case "14":
        if (!hasPrograms || !hasSubjects) Console.WriteLine("\nAd
d programs and subjects first.");
        else ProgramView.AddSubjectToProgram();
        break;
    case "15":
        if (!hasStudentsWithPrefs) Console.WriteLine("\nNo studen
ts with program preferences.");
        else AdmissionView.GenerateMeritList();
        break;
    case "16":
        if (!hasStudentsWithPrefs || !hasAvailablePrograms) Conso
le.WriteLine("\nCannot process admission. Add students with preferences and a
vailable programs first.");
        else AdmissionView.ProcessAdmission();
        break;
    case "17":
        if (!hasAdmittedStudents) Console.WriteLine("\nNo admitte
d students.");
        else AdmissionView.ViewAdmittedStudents();
        break;
    case "18":
        if (!hasAdmittedStudents) Console.WriteLine("\nNo admitte
d students.");
        else AdmissionView.ViewAllocationDetails();
        break;
    case "19":
        if (!hasAdmittedStudents) Console.WriteLine("\nNo admitte
d students.");
        else AdmissionView.RegisterSubjects();
        break;
    case "20":
        if (!anyRegisteredSubjects) Console.WriteLine("\nNo regis
tered subjects yet.");
        else AdmissionView.ViewRegisteredSubjects();
        break;
    case "21":
        if (!anyRegisteredSubjects) Console.WriteLine("\nNo regis
tered subjects. Register subjects first.");
        else AdmissionView.ViewFee();
        break;
    case "22":
        if (!anyRegisteredSubjects) Console.WriteLine("\nNo regis
tered subjects.");
        else AdmissionView.ViewAllFees();
        break;
    case "0":
```

```
        Console.WriteLine("\nThank you for using the system. Good  
bye!");  
        Environment.Exit(0);  
        break;  
    default:  
        Console.WriteLine("\nInvalid choice. Please try again.");  
        break;  
    }  
  
    Console.WriteLine("\nPress Enter to continue...");  
    Console.ReadLine();  
}  
}  
}
```

ProgramView

```
using System;  
using System.Collections.Generic;  
namespace UniversityAdmissionSystem  
{  
    public class ProgramView  
    {  
        public static void AddProgram()  
        {  
            try  
            {  
                Console.WriteLine("\n===== ADD NEW PROGRAM =====");  
                Console.WriteLine("  b. Back to Main Menu\n");  
  
                string programId = InputValidator.GetValidString("Enter Progr  
am ID: ", false, true);  
                if (Program.GetById(programId) != null)  
                {  
                    Console.WriteLine("Error: Program with this ID already ex  
ists.");  
                    return;  
                }  
  
                string degreeTitle = InputValidator.GetValidString("Enter Deg  
ree Title: ", false, true);  
                string department = InputValidator.GetValidString("Enter Depa  
rtment: ", false, true);  
                int duration = InputValidator.GetValidInteger("Enter Duration  
(years): ", 2, 6, null, true);  
                int seats = InputValidator.GetValidInteger("Enter Total Seats  
: ", 1, 500, null, true);  
  
                Program program = new Program(programId, degreeTitle, departm  
ent, duration, seats);  
  
                if (Program.Add(program))  
                {  
                    Console.WriteLine("Program added successfully!");  
                }  
                else  
                {  
                    Console.WriteLine("Error: Could not add program.");  
                }  
            }  
            catch { }  
        }  
    }  
}
```

```
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void ViewAllPrograms()
{
    List<Program> programs = Program.GetAll();
    if (programs.Count == 0)
    {
        Console.WriteLine("\nNo programs found.");
        return;
    }

    Console.WriteLine("\n===== ALL PROGRAMS =====");
    Console.WriteLine("ID      | Degree Title      | Department      |
Duration | Total Seats | Available");
    Console.WriteLine("-----|-----|-----");
    |-----|-----|-----");
    foreach (Program p in programs)
    {
        Console.WriteLine($"{p.ProgramId,-4} | {p.DegreeTitle,-
19} | {p.Department,-
13} | {p.Duration,8} | {p.TotalSeats,11} | {p.AvailableSeats}");
    }
    Console.WriteLine("===== \n");
}

public static void AddSubject()
{
    try
    {
        Console.WriteLine("\n===== ADD NEW SUBJECT =====");
        Console.WriteLine(" b. Back to Main Menu\n");

        string subjectCode = InputValidator.GetValidString("Enter Sub
ject Code: ", false, true);
        if (Subject.GetByCode(subjectCode) != null)
        {
            Console.WriteLine("Error: Subject with this code already
exists.");
            return;
        }

        string name = InputValidator.GetValidString("Enter Subject Na
me: ", false, true);
        int creditHours = InputValidator.GetValidInteger("Enter Credi
t Hours (1-6): ", 1, 6, null, true);
        string subjectType = InputValidator.GetValidChoice("Enter Sub
ject Type (Core/Elective): ", new[] { "Core", "Elective" }, true);
        double feePerCreditHour = InputValidator.GetValidDouble("Ente
r Fee Per Credit Hour: ", 0, 10000, null, true);

        Subject subject = new Subject(subjectCode, name, creditHours,
subjectType, feePerCreditHour);

        if (Subject.Add(subject))
        {
```

```

        Console.WriteLine("Subject added successfully!");
    }
    else
    {
        Console.WriteLine("Error: Could not add subject.");
    }
}
catch (NavigationBackException)
{
    Console.WriteLine("\nOperation cancelled.");
}
}

public static void ViewAllSubjects()
{
    List<Subject> subjects = Subject.GetAll();
    if (subjects.Count == 0)
    {
        Console.WriteLine("\nNo subjects found.");
        return;
    }

    Console.WriteLine("\n===== ALL SUBJECTS =====");
    Console.WriteLine("Code    | Subject Name          | Credit Hours
| Type    | Fee/CH");
    Console.WriteLine("-----|-----|-----");
    Console.WriteLine("-----|-----");
    foreach (Subject s in subjects)
    {
        Console.WriteLine($"{s.SubjectCode,-6} | {s.Name,-
19} | {s.CreditHours,12} | {s.SubjectType,-8} | {s.FeePerCreditHour,7:F2}");
    }
    Console.WriteLine("===== \n");
}

public static void EditSubject()
{
    try
    {
        Console.WriteLine("\n===== EDIT SUBJECT =====");
        Console.WriteLine("    b. Back to Main Menu\n");

        List<Subject> subjects = Subject.GetAll();
        if (subjects.Count == 0)
        {
            Console.WriteLine("No subjects found.");
            return;
        }

        Console.WriteLine("Available Subjects:");
        Console.WriteLine("Code    | Subject Name          | Credit Ho
urs | Fee/CH");
        Console.WriteLine("-----|-----|-----");
        Console.WriteLine("-----|-----");
        foreach (var s in subjects)
        {
            Console.WriteLine($"{s.SubjectCode,-6} | {s.Name,-
19} | {s.CreditHours,12} | {s.FeePerCreditHour,7:F2}");
        }
    }
}

```

```
        string subjectCode = InputValidator.GetValidString("\nEnter S
subject Code to edit: ", false, true);
        Subject subject = Subject.GetByCode(subjectCode);

        if (subject == null)
        {
            Console.WriteLine("Subject not found.");
            return;
        }

        Console.WriteLine($"Editing Subject: {subject.Name} (Code:
{subject.SubjectCode})");
        Console.WriteLine("Press Enter to keep current values.\n");

        string name = InputValidator.GetValidString($"Enter Subject N
ame [{subject.Name}]: ", true, true);
        if (!string.IsNullOrEmpty(name)) subject.Name = name;

        subject.CreditHours = InputValidator.GetValidInteger($"Enter
Credit Hours (1-
6) [{subject.CreditHours}]: ", 1, 6, subject.CreditHours, true);

        string subjectType;
        while (true)
        {
            subjectType = InputValidator.GetValidString($"Enter Subje
ct Type (Core/Elective) [{subject.SubjectType}]: ", true, true);
            if (string.IsNullOrEmpty(subjectType)) break;
            if (subjectType.Equals("Core", StringComparison.OrdinalIg
noreCase) || subjectType.Equals("Elective", StringComparison.OrdinalIg
noreCase))
            {
                subject.SubjectType = subjectType;
                break;
            }
            Console.WriteLine("Error: Please enter Core or Elective."
);
        }

        subject.FeePerCreditHour = InputValidator.GetValidDouble($"En
ter Fee Per Credit Hour [{subject.FeePerCreditHour}]: ", 0, 10000, subject.Fe
ePerCreditHour, true);

        Console.WriteLine("Subject updated successfully!");
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void DeleteSubject()
{
    try
    {
        Console.WriteLine("\n===== DELETE SUBJECT =====");
        Console.WriteLine("  b. Back to Main Menu\n");

        List<Subject> subjects = Subject.GetAll();
        if (subjects.Count == 0)
```

```

        {
            Console.WriteLine("No subjects found.");
            return;
        }

        Console.WriteLine("Available Subjects:");
        Console.WriteLine("Code    | Subject Name          | Credit Ho
urs");
        Console.WriteLine("-----|-----|-----
-----");
        foreach (var s in subjects)
        {
            Console.WriteLine($"{s.SubjectCode,-6} | {s.Name,-
19} | {s.CreditHours,12}");
        }

        string subjectCode = InputValidator.GetValidString("\nEnter S
ubject Code to delete: ", false, true);
        Subject subject = Subject.GetByCode(subjectCode);

        if (subject == null)
        {
            Console.WriteLine("Subject not found.");
            return;
        }

        Console.WriteLine($"{subject.Name} (Code: {subject.SubjectCode})?");
        if (InputValidator.GetYesNo("Confirm (y/n): ", true))
        {
            if (Subject.Delete(subjectCode))
            {
                Console.WriteLine("Subject deleted successfully!");
            }
            else
            {
                Console.WriteLine("Error: Could not delete subject.")
            }
        }
        else
        {
            Console.WriteLine("Deletion cancelled.");
        }
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void AddSubjectToProgram()
{
    try
    {
        Console.WriteLine("\n===== ADD SUBJECT TO PROGRAM =====
=====");
        Console.WriteLine("    b. Back to Main Menu\n");
        List<Program> programs = Program.GetAll();
    }
}

```

```
        if (programs.Count == 0)
        {
            Console.WriteLine("No programs available.");
            return;
        }

        Console.WriteLine("Available Programs:");
        Console.WriteLine("ID      | Degree Title      | Department
| Seats");
        Console.WriteLine("-----|-----|-----");
        foreach (var p in programs)
        {
            Console.WriteLine($"{p.ProgramId,-4} | {p.DegreeTitle,-
19} | {p.Department,-13} | {p.TotalSeats}");
        }

        string programId = InputValidator.GetValidString("\nEnter Pro
gram ID: ", false, true);
        Program program = Program.GetById(programId);

        if (program == null)
        {
            Console.WriteLine("Program not found. Use one of the IDs
shown above.");
            return;
        }

        if (program.GetTotalCreditHours() >= Program.MaxCreditHours)
        {
            Console.WriteLine($"Program already has maximum {Program.
MaxCreditHours} credit hours.");
            return;
        }

        Console.WriteLine("Available subjects:");
        List<Subject> allSubjects = Subject.GetAll();
        if (allSubjects.Count == 0)
        {
            Console.WriteLine("No subjects available. Add subjects fi
rst.");
            return;
        }

        foreach (Subject s in allSubjects)
        {
            Console.WriteLine($" {s.SubjectCode}: {s.Name} ({s.Credi
tHours} CH)");
        }

        string subjectCode = InputValidator.GetValidString("\nEnter S
ubject Code to add: ", false, true);
        Subject subject = Subject.GetByCode(subjectCode);

        if (subject == null)
        {
            Console.WriteLine("Subject not found.");
            return;
        }
    }
```

```
        if (!program.CanAddSubject(subject))
        {
            Console.WriteLine($"Cannot add. This would exceed {Program.MaxCreditHours} credit hours limit.");
            return;
        }

        program.AddSubject(subject);
        Console.WriteLine("Subject added to program successfully!");
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void EditProgram()
{
    try
    {
        Console.WriteLine("\n===== EDIT PROGRAM =====");
        Console.WriteLine("    b. Back to Main Menu\n");

        List<Program> programs = Program.GetAll();
        if (programs.Count == 0)
        {
            Console.WriteLine("No programs found.");
            return;
        }

        Console.WriteLine("Available Programs:");
        Console.WriteLine("ID      | Degree Title      | Department
        | Seats");
        Console.WriteLine("-----|-----|-----");
        foreach (var p in programs)
        {
            Console.WriteLine($"{p.ProgramId,-4} | {p.DegreeTitle,-
19} | {p.Department,-13} | {p.TotalSeats}");
        }

        string programId = InputValidator.GetValidString("\nEnter Program ID to edit: ", false, true);
        Program program = Program.GetById(programId);

        if (program == null)
        {
            Console.WriteLine("Program not found.");
            return;
        }

        Console.WriteLine($"Editing Program: {program.DegreeTitle} (ID: {program.ProgramId})");
        Console.WriteLine("Press Enter to keep current values.\n");

        string degreeTitle = InputValidator.GetValidString($"Enter Degree Title [{program.DegreeTitle}]: ", true, true);
        if (!string.IsNullOrEmpty(degreeTitle)) program.DegreeTitle = degreeTitle;
    }
}
```

```

        string department = InputValidator.GetValidString($"Enter Department [{program.Department}]: ", true, true);
        if (!string.IsNullOrEmpty(department)) program.Department = department;

        program.Duration = InputValidator.GetValidInteger($"Enter Duration (2-6) [{program.Duration}]: ", 2, 6, program.Duration, true);
        program.TotalSeats = InputValidator.GetValidInteger($"Enter Total Seats [{program.TotalSeats}]: ", 1, 500, program.TotalSeats, true);

        Console.WriteLine("Program updated successfully!");
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void DeleteProgram()
{
    try
    {
        Console.WriteLine("\n===== DELETE PROGRAM =====");
        Console.WriteLine("    b. Back to Main Menu\n");

        List<Program> programs = Program.GetAll();
        if (programs.Count == 0)
        {
            Console.WriteLine("No programs found.");
            return;
        }

        Console.WriteLine("Available Programs:");
        Console.WriteLine("ID      | Degree Title      | Available Seats");
        Console.WriteLine("-----|-----");
        foreach (var p in programs)
        {
            Console.WriteLine($"{p.ProgramId,-4} | {p.DegreeTitle,-19} | {p.AvailableSeats}");
        }

        string programId = InputValidator.GetValidString("\nEnter Program ID to delete: ", false, true);
        Program program = Program.GetById(programId);

        if (program == null)
        {
            Console.WriteLine("Program not found.");
            return;
        }

        Console.WriteLine($"Are you sure you want to delete {program.DegreeTitle} (ID: {program.ProgramId})?");
        if (InputValidator.GetYesNo("Confirm (y/n): ", true))
        {
            if (Program.Delete(programId))
            {
                Console.WriteLine("Program deleted successfully!");
            }
        }
    }
    catch { }
}

```

```
        }
        else
        {
            Console.WriteLine("Error: Could not delete program.")
        }
    }
    else
    {
        Console.WriteLine("Deletion cancelled.");
    }
}
catch (NavigationBackException)
{
    Console.WriteLine("\nOperation cancelled.");
}
}
}
}
```

StudentView

```
using System;
using System.Collections.Generic;
namespace UniversityAdmissionSystem
{
    public class StudentView
    {
        public static void AddStudent()
        {
            try
            {
                Console.WriteLine("\n===== ADD NEW STUDENT =====");
                Console.WriteLine("    b. Back to Main Menu\n");

                string studentId = InputValidator.GetValidString("Enter Student ID: ", false, true);
                if (Student.GetById(studentId) != null)
                {
                    Console.WriteLine("Error: Student with this ID already exists.");
                    return;
                }

                string name = InputValidator.GetValidString("Enter Student Name: ", false, true);
                int age = InputValidator.GetValidInteger("Enter Age (15-25): ", 15, 25, null, true);
                double fscMarks = InputValidator.GetValidDouble("Enter FSC Marks (0-1200): ", 0, 1200, null, true);
                double ecatMarks = InputValidator.GetValidDouble("Enter ECAT Marks (0-400): ", 0, 400, null, true);

                Student student = new Student(studentId, name, age, fscMarks, ecatMarks);

                if (Student.Add(student))
                {
                    Console.WriteLine("Student added successfully!");
                }
            }
            catch { }
        }
    }
}
```

```

        else
        {
            Console.WriteLine("Error: Could not add student.");
        }
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void ViewAllStudents()
{
    List<Student> students = Student.GetAll();
    if (students.Count == 0)
    {
        Console.WriteLine("\nNo students found.");
        return;
    }

    Console.WriteLine("\n===== ALL STUDENTS =====");
    Console.WriteLine("ID          | Name          | Age | FSC    | ECAT
| Status");
    Console.WriteLine("-----|-----|----|-----|-----
---|-----");
    foreach (Student s in students)
    {
        string status = s.IsAdmitted ? "Admitted" : "Not Admitted";
        Console.WriteLine($"{s.StudentId,-7} | {s.Name,-
13} | {s.Age,3} | {s.FscMarks,6:F2} | {s.EcatMarks,6:F2} | {status}");
    }
    Console.WriteLine("=====\\n");
}

public static void AddProgramPreferences()
{
    try
    {
        Console.WriteLine("\n===== ADD PROGRAM PREFERENCES =====
=====");
        Console.WriteLine("  b. Back to Main Menu\\n");

        List<Student> allStudents = Student.GetAll();
        if (allStudents.Count == 0)
        {
            Console.WriteLine("No students found. Add a student first
.");
            return;
        }

        Console.WriteLine("Available Students:");
        Console.WriteLine("ID          | Name          | Age | Merit Scor
e");
        Console.WriteLine("-----|-----|----|-----
-");
        foreach (var s in allStudents)
        {
            Console.WriteLine($"{s.StudentId,-6} | {s.Name,-
13} | {s.Age,3} | {s.CalculateMerit():F2}");
        }
    }
    catch { }
}

```

```
        string studentId = InputValidator.GetValidString("\nEnter Student ID: ", false, true);
        Student student = Student.GetById(studentId);

        if (student == null)
        {
            Console.WriteLine("Student not found.");
            Console.WriteLine("Available Student IDs: " + string.Join(", ", allStudents.ConvertAll(s => s.StudentId)));
            return;
        }

        if (student.Preferences.Count > 0)
        {
            if (!InputValidator.GetYesNo("Do you want to replace them? (y/n): ", true))
            {
                return;
            }
            student.Preferences.Clear();
        }

        List<Program> programs = Program.GetAll();
        if (programs.Count == 0)
        {
            Console.WriteLine("No programs available. Please add programs first.");
            return;
        }

        Console.WriteLine("\nAvailable Programs:");
        Console.WriteLine("ID      | Program Name      | Department | Seats");
        Console.WriteLine("-----|-----");
        foreach (var p in programs)
        {
            Console.WriteLine($"{p.ProgramId,-4} | {p.DegreeTitle,-23} | {p.Department,-13} | {p.TotalSeats}");
        }

        string input = InputValidator.GetValidString("\nEnter program IDs separated by commas (e.g., P1,P2,P3):\n", false, true);
        string[] prefs = input.Split(',');

        List<string> invalidIds = new List<string>();
        foreach (string pref in prefs)
        {
            string trimmed = pref.Trim();
            if (Program.GetById(trimmed) != null)
            {
                student.AddPreference(trimmed);
            }
            else
            {
                invalidIds.Add(trimmed);
            }
        }

        Console.WriteLine($"Added {student.Preferences.Count} program preferences.");
```

```

        if (invalidIds.Count > 0)
        {
            Console.WriteLine($"Warning: The following program IDs we
re not found and were skipped: {string.Join(", ", invalidIds)}");
        }
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void EditStudent()
{
    try
    {
        Console.WriteLine("\n===== EDIT STUDENT =====");
        Console.WriteLine("    b. Back to Main Menu\n");

        List<Student> students = Student.GetAll();
        if (students.Count == 0)
        {
            Console.WriteLine("No students found.");
            return;
        }

        Console.WriteLine("Available Students:");
        Console.WriteLine("ID          | Name          | Age | FSC      | E
CAT");
        Console.WriteLine("-----|-----|----|-----|--
-----");
        foreach (var s in students)
        {
            Console.WriteLine($"{s.StudentId,-6} | {s.Name,-
13} | {s.Age,3} | {s.FscMarks,6:F2} | {s.EcatMarks,6:F2}");
        }

        string studentId = InputValidator.GetValidString("\nEnter Stu
dent ID to edit: ", false, true);
        Student student = Student.GetById(studentId);

        if (student == null)
        {
            Console.WriteLine("Student not found.");
            return;
        }

        Console.WriteLine($"Editing Student: {student.Name} (ID: {s
tudent.StudentId})");
        Console.WriteLine("Press Enter to keep current values.\n");

        string name = InputValidator.GetValidString($"Enter Name [{st
udent.Name}]: ", true, true);
        if (!string.IsNullOrEmpty(name)) student.Name = name;

        student.Age = InputValidator.GetValidInteger($"Enter Age (15-
25) [{student.Age}]: ", 15, 25, student.Age, true);
        student.FscMarks = InputValidator.GetValidDouble($"Enter FSC
Marks (0-1200) [{student.FscMarks}]: ", 0, 1200, student.FscMarks, true);
    }
}

```

```

        student.EcatMarks = InputValidator.GetValidDouble($"Enter ECA
T Marks (0-400) [{student.EcatMarks}]: ", 0, 400, student.EcatMarks, true);

        Console.WriteLine("Student updated successfully!");
    }
    catch (NavigationBackException)
    {
        Console.WriteLine("\nOperation cancelled.");
    }
}

public static void DeleteStudent()
{
    try
    {
        Console.WriteLine("\n===== DELETE STUDENT =====");
        Console.WriteLine("  b. Back to Main Menu\n");

        List<Student> students = Student.GetAll();
        if (students.Count == 0)
        {
            Console.WriteLine("No students found.");
            return;
        }

        Console.WriteLine("Available Students:");
        Console.WriteLine("ID          | Name          | Age | Status");
        Console.WriteLine("-----|-----|----|-----");

        foreach (var s in students)
        {
            string status = s.IsAdmitted ? "Admitted" : "Not Admitted";

            Console.WriteLine($"{s.StudentId,-6} | {s.Name,-
13} | {s.Age,3} | {status}");
        }

        string studentId = InputValidator.GetValidString("\nEnter Stu
dent ID to delete: ", false, true);
        Student student = Student.GetById(studentId);

        if (student == null)
        {
            Console.WriteLine("Student not found.");
            return;
        }

        Console.WriteLine($"Are you sure you want to delete {student
t.Name} (ID: {student.StudentId})?");
        if (InputValidator.GetYesNo("Confirm (y/n): ", true))
        {
            if (Student.Delete(studentId))
            {
                Console.WriteLine("Student deleted successfully!");
            }
            else
            {
                Console.WriteLine("Error: Could not delete student.")
            }
        }
    }
}

```

```
        }  
        else  
        {  
            Console.WriteLine("Deletion cancelled.");  
        }  
    }  
    catch (NavigationBackException)  
    {  
        Console.WriteLine("\nOperation cancelled.");  
    }  
    }  
    }  
}
```

2. Business Logic Layer (Core System)

Handles all processing and rules

Classes

- Student
- Program
- Subject
- AdmissionManager
- MeritCalculator
- RegistrationManager
- FeeCalculator

Responsibilities

- Store and process student/program data
- Calculate merit based on FSC & ECAT
- Generate merit lists
- Allocate students to programs
- Enforce constraints:
 - Max 9 credit hours per student 20 credit hours per program
- Handle subject registration
- Calculate fees

Example Logic

- Sort students by merit
- Assign program based on preference + merit
- Check credit hour limits

Code:

AdmissionManager

```
using System;
using System.Collections.Generic;
namespace UniversityAdmissionSystem
{
    public class AdmissionManager
    {
        public static string ProcessAdmission()
        {
            List<Student> meritList = MeritCalculator.GenerateMeritList();
            if (meritList.Count == 0)
            {
                return "No students with program preferences found.";
            }

            int admittedCount = 0;
            foreach (Student student in meritList)
            {
                foreach (string prefId in student.Preferences)
                {
                    Program program = ProgramDL.GetProgramById(prefId);
                    if (program != null && program.HasAvailableSeats())
                    {
                        student.AllocatedProgram = program;
                        student.IsAdmitted = true;
                        program.AllocateSeat();
                        admittedCount++;
                        break;
                    }
                }
            }

            return $"Admission process completed. {admittedCount} students ad
mitted.";
        }

        public static void DisplayAdmittedStudents()
        {
            List<Student> admitted = StudentDL.GetAdmittedStudents();
            if (admitted.Count == 0)
            {
                Console.WriteLine("\nNo students have been admitted yet.");
                return;
            }

            Console.WriteLine("\n===== ADMITTED STUDENTS =====");
            Console.WriteLine("Student ID | Name | Program
| Merit");
            Console.WriteLine("-----|-----|-----
-----|-----");
            foreach (Student s in admitted)
            {
                string programName = s.AllocatedProgram != null ? s.Allocated
Program.DegreeTitle : "N/A";
                Console.WriteLine($"{s.StudentId,-9} | {s.Name,-
13} | {programName,-24} | {s.CalculateMerit(),5:F2}");
            }
            Console.WriteLine("===== \n");
        }

        public static void DisplayAllocationDetails()
    }
}
```

```
    {
        List<Student> admitted = StudentDL.GetAdmittedStudents();
        if (admitted.Count == 0)
        {
            Console.WriteLine("\nNo students have been admitted yet.");
            return;
        }

        Console.WriteLine("\n===== PROGRAM ALLOCATION DETAILS =====");
        foreach (Student s in admitted)
        {
            Console.WriteLine($"Student: {s.Name} (ID: {s.StudentId})");
            Console.WriteLine($"    Merit Score: {s.CalculateMerit():F2}");
            Console.WriteLine($"    Allocated Program: {(s.AllocatedProgram != null ? s.AllocatedProgram.DegreeTitle : "Not Allocated")}");
            Console.WriteLine($"    Department: {(s.AllocatedProgram != null ? s.AllocatedProgram.Department : "N/A")}");
            Console.WriteLine($"    Duration: {(s.AllocatedProgram != null ? s.AllocatedProgram.Duration + " years" : "N/A")}");
        }
        Console.WriteLine("\n=====");
    }
}
```

FeeCalculator

```
using System;
using System.Collections.Generic;
namespace UniversityAdmissionSystem
{
    public class FeeCalculator
    {
        public static double CalculateFee(string studentId)
        {
            Student student = StudentDL.GetStudentById(studentId);
            if (student == null || student.RegisteredSubjects.Count == 0)
            {
                return 0;
            }

            double totalFee = 0;
            foreach (Subject subject in student.RegisteredSubjects)
            {
                totalFee += subject.GetTotalFee();
            }
            return totalFee;
        }

        public static void DisplayFeeDetails(string studentId)
        {
            Student student = StudentDL.GetStudentById(studentId);
            if (student == null)
            {
                Console.WriteLine("Student not found.");
                return;
            }
        }
    }
}
```

```

    }

    if (student.RegisteredSubjects.Count == 0)
    {
        Console.WriteLine("\nNo subjects registered. Fee cannot be calculated.");
        return;
    }

    Console.WriteLine($"\\n===== FEE DETAILS FOR {student.Name} =====");
    Console.WriteLine($"Student ID: {student.StudentId}");
    Console.WriteLine($"Program: {(student.AllocatedProgram != null ? student.AllocatedProgram.DegreeTitle : "N/A")}");
    Console.WriteLine("\\nSubject Fee Breakdown:");
    Console.WriteLine("Code | Subject Name | Credit Hours | Rate | Amount");
    Console.WriteLine("-----|-----|-----|-----|-----");
    Console.WriteLine("-----|-----");

    double total = 0;
    foreach (Subject s in student.RegisteredSubjects)
    {
        double amount = s.CreditHours * s.FeePerCreditHour;
        total += amount;
        Console.WriteLine($"{s.SubjectCode,-6} | {s.Name,-19} | {s.CreditHours,12} | {s.FeePerCreditHour,7:F2} | {amount,7:F2}");
    }

    Console.WriteLine("-----");
    Console.WriteLine($"Total Fee: {total,39:F2}");
    Console.WriteLine("=====\\n");
}

public static void DisplayAllAdmittedStudentsFee()
{
    List<Student> admitted = StudentDL.GetAdmittedStudents();
    if (admitted.Count == 0)
    {
        Console.WriteLine("\\nNo admitted students found.");
        return;
    }

    Console.WriteLine("\\n===== ALL STUDENTS FEE OVERVIEW =====");
    Console.WriteLine("Student ID | Name | Subjects | Credit s | Total Fee");
    Console.WriteLine("-----|-----|-----|-----|-----");
    Console.WriteLine("-----|-----");
    foreach (Student s in admitted)
    {
        int credits = s.GetTotalCreditHours();
        double fee = CalculateFee(s.StudentId);
        Console.WriteLine($"{s.StudentId,-9} | {s.Name,-19} | {s.RegisteredSubjects.Count,8} | {credits,7} | {fee,9:F2}");
    }
    Console.WriteLine("=====\\n");
}

```

```
    }  
  }  
}
```

MeritCalculator

```
using System;  
using System.Collections.Generic;  
namespace UniversityAdmissionSystem  
{  
    public class MeritCalculator  
    {  
        public static List<Student> GenerateMeritList()  
        {  
            List<Student> studentsWithPrefs = new List<Student>(StudentDL.Get  
StudentsWithPreferences());  
            studentsWithPrefs.Sort((s1, s2) =>  
            {  
                int result = s2.CalculateMerit().CompareTo(s1.CalculateMerit(  
));  
                if (result == 0)  
                    return s2.EcatMarks.CompareTo(s1.EcatMarks);  
                return result;  
            });  
            return studentsWithPrefs;  
        }  
  
        public static double CalculateMerit(double fscMarks, double ecatMarks  
)  
        {  
            return (fscMarks * 0.6) + (ecatMarks * 0.4);  
        }  
  
        public static void DisplayMeritList()  
        {  
            List<Student> meritList = GenerateMeritList();  
            if (meritList.Count == 0)  
            {  
                Console.WriteLine("\nNo students with program preferences fou  
nd.");  
                return;  
            }  
  
            Console.WriteLine("\n===== MERIT LIST =====");  
            Console.WriteLine("Rank | Student ID | Name | FSC | E  
CAT | Merit");  
            Console.WriteLine("-----|-----|-----|-----|--  
-----|-----");  
            for (int i = 0; i < meritList.Count; i++)  
            {  
                Student s = meritList[i];  
                Console.WriteLine($"{i + 1,4} | {s.StudentId,-10} | {s.Name,-  
13} | {s.FscMarks,6:F2} | {s.EcatMarks,6:F2} | {s.CalculateMerit(),5:F2}");  
            }  
            Console.WriteLine("=====\n");  
        }  
    }  
}
```

Program

```
using System;
using System.Collections.Generic;
namespace UniversityAdmissionSystem
{
    public class Program
    {
        private string programId;
        private string degreeTitle;
        private string department;
        private int duration;
        private int totalSeats;
        private int availableSeats;
        private List<Subject> subjects;

        public Program()
        {
            subjects = new List<Subject>();
            totalSeats = 0;
            availableSeats = 0;
        }

        public Program(string programId, string degreeTitle, string department, int duration, int totalSeats)
        {
            this.programId = programId;
            this.degreeTitle = degreeTitle;
            this.department = department;
            this.duration = duration;
            this.totalSeats = totalSeats;
            this.availableSeats = totalSeats;
            this.subjects = new List<Subject>();
        }

        public string ProgramId { get => programId; set => programId = value; }
        public string DegreeTitle { get => degreeTitle; set => degreeTitle = value; }
        public string Department { get => department; set => department = value; }
        public int Duration { get => duration; set => duration = value; }
        public int TotalSeats
        {
            get => totalSeats;
            set
            {
                int allocated = totalSeats - availableSeats;
                totalSeats = value;
                availableSeats = Math.Max(0, value - allocated);
            }
        }
        public int AvailableSeats { get => availableSeats; }
        public List<Subject> Subjects { get => subjects; }

        public bool HasAvailableSeats()
        {
            return availableSeats > 0;
        }
    }
}
```

```
        public void AllocateSeat()
        {
            if (availableSeats > 0)
                availableSeats--;
        }

        public void ReleaseSeat()
        {
            if (availableSeats < totalSeats)
                availableSeats++;
        }

        public const int MaxCreditHours = 20;

        public void AddSubject(Subject subject)
        {
            if (GetTotalCreditHours() + subject.CreditHours <= MaxCreditHours
)
            {
                subjects.Add(subject);
            }
        }

        public bool CanAddSubject(Subject subject)
        {
            return GetTotalCreditHours() + subject.CreditHours <= MaxCreditHo
urs;
        }

        public void RemoveSubject(Subject subject)
        {
            subjects.Remove(subject);
        }

        public int GetTotalCreditHours()
        {
            int total = 0;
            foreach (var subject in subjects)
            {
                total += subject.CreditHours;
            }
            return total;
        }

        public override string ToString()
        {
            return $"{degreeTitle} - {department} (Seats: {availableSeats}/{t
otalSeats})";
        }

        public static List<Program> GetAll()
        {
            return ProgramDL.GetAllPrograms();
        }

        public static Program GetById(string programId)
        {
            return ProgramDL.GetProgramById(programId);
        }
    }
}
```

```
        public static bool Add(Program program)
        {
            return ProgramDL.AddProgram(program);
        }

        public static bool Delete(string programId)
        {
            return ProgramDL.DeleteProgram(programId);
        }

        public static List<Program> GetAvailable()
        {
            return ProgramDL.GetAvailablePrograms();
        }

        public static bool Any()
        {
            return ProgramDL.GetAllPrograms().Count > 0;
        }

        public static bool AnyAvailable()
        {
            return ProgramDL.GetAvailablePrograms().Count > 0;
        }
    }
}
```

RegistrationManager

```
using System;
using System.Collections.Generic;
namespace UniversityAdmissionSystem
{
    public class RegistrationManager
    {
        public static string RegisterSubjects(string studentId, List<string>
subjectCodes)
        {
            Student student = StudentDL.GetStudentById(studentId);
            if (student == null)
            {
                return "Student not found.";
            }

            if (!student.IsAdmitted)
            {
                return "Student is not admitted to any program.";
            }

            List<Subject> programSubjects = student.AllocatedProgram.Subjects
;
            List<Subject> newSubjects = new List<Subject>();
            int totalCredits = 0;
            int maxCredits = student.GetMaxCreditHours();
            List<string> skippedSubjects = new List<string>();
            List<string> invalidCodes = new List<string>();

            foreach (string code in subjectCodes)
```

```
{
    Subject subject = SubjectDL.GetSubjectByCode(code);
    if (subject == null)
    {
        invalidCodes.Add(code);
        continue;
    }

    bool belongsToProgram = false;
    foreach (var progSubject in programSubjects)
    {
        if (progSubject.SubjectCode.ToLower() == code.ToLower())
        {
            belongsToProgram = true;
            break;
        }
    }

    if (!belongsToProgram)
    {
        skippedSubjects.Add($"{subject.Name} (not in {student.All
ocatedProgram.DegreeTitle})");
        continue;
    }

    if (newSubjects.Exists(s => s.SubjectCode.Equals(code, String
Comparison.OrdinalIgnoreCase)))
    {
        skippedSubjects.Add($"{subject.Name} (duplicate)");
        continue;
    }

    if (totalCredits + subject.CreditHours <= maxCredits)
    {
        totalCredits += subject.CreditHours;
        newSubjects.Add(subject);
    }
    else
    {
        skippedSubjects.Add($"{subject.Name} (exceeds credit limi
t)");
    }
}

student.RegisteredSubjects.Clear();
student.RegisteredSubjects.AddRange(newSubjects);

string msg = "";
if (invalidCodes.Count > 0)
{
    msg += $"Invalid codes skipped: {string.Join(", ", invalidCod
es)}. ";
}

if (skippedSubjects.Count > 0)
{
    msg += $"Registered {newSubjects.Count} subjects ({totalCredi
ts} credits). Skipped: {string.Join(", ", skippedSubjects)}. Max allowed: {ma
xCredits} credits.";
}
```

```

        else
        {
            msg += $"Successfully registered {student.RegisteredSubjects.
Count} subjects ({totalCredits} credit hours).";
        }

        return msg;
    }

    public static void DisplayRegisteredSubjects(string studentId)
    {
        Student student = StudentDL.GetStudentById(studentId);
        if (student == null)
        {
            Console.WriteLine("Student not found.");
            return;
        }

        if (student.RegisteredSubjects.Count == 0)
        {
            Console.WriteLine("\nNo subjects registered for this student.
");
            return;
        }

        Console.WriteLine($"\\n===== REGISTERED SUBJECTS FOR {student
.Name} =====");
        Console.WriteLine("Code    | Subject Name          | Credit Hours
| Type    | Fee");
        Console.WriteLine("-----|-----|-----
|-----|-----");
        foreach (Subject s in student.RegisteredSubjects)
        {
            Console.WriteLine($"{s.SubjectCode,-6} | {s.Name,-
19} | {s.CreditHours,12} | {s.SubjectType,-7} | {s.GetTotalFee(),7:F2}");
        }
        Console.WriteLine($"Total Credit Hours: {student.GetTotalCreditHo
urs()}");
        Console.WriteLine($"Max Allowed: {student.GetMaxCreditHours()}");
        Console.WriteLine("=====\\n");
    }

    public static bool CanAddSubject(Student student, Subject subject)
    {
        return student.GetTotalCreditHours() + subject.CreditHours <= stu
dent.GetMaxCreditHours();
    }
}

```

3. Data Layer (Storage Layer)

Handles data storage (in-memory using lists/arrays)

Classes

- StudentDL

- ProgramDL
- SubjectDL

Responsibilities

- Store:
 - List of students
 - List of programs
 - List of subjects
- Provide methods:
 - Add data
 - Retrieve data
 - Search records

Code:

StudentDL

```
using System;
using System.Collections.Generic;
namespace UniversityAdmissionSystem
{
    public class StudentDL
    {
        private static List<Student> students = new List<Student>();

        public static bool AddStudent(Student student)
        {
            if (GetStudentById(student.StudentId) == null)
            {
                students.Add(student);
                return true;
            }
            return false;
        }

        public static List<Student> GetAllStudents()
        {
            return students;
        }

        public static Student GetStudentById(string studentId)
        {
            foreach (var student in students)
            {
                if (student.StudentId.ToLower() == studentId.ToLower())
                    return student;
            }
            return null;
        }

        public static List<Student> GetAdmittedStudents()
        {
            List<Student> admitted = new List<Student>();
        }
    }
}
```

```
        foreach (var student in students)
        {
            if (student.IsAdmitted)
                admitted.Add(student);
        }
        return admitted;
    }

    public static List<Student> GetStudentsWithPreferences()
    {
        List<Student> withPrefs = new List<Student>();
        foreach (var student in students)
        {
            if (student.Preferences.Count > 0)
                withPrefs.Add(student);
        }
        return withPrefs;
    }

    public static bool UpdateStudent(Student student)
    {
        for (int i = 0; i < students.Count; i++)
        {
            if (students[i].StudentId.ToLower() == student.StudentId.ToLo
wer())
            {
                students[i] = student;
                return true;
            }
        }
        return false;
    }

    public static bool DeleteStudent(string studentId)
    {
        Student student = GetStudentById(studentId);
        if (student != null)
        {
            students.Remove(student);
            return true;
        }
        return false;
    }
}
```

ProgramDL

```
using System;
using System.Collections.Generic;
namespace UniversityAdmissionSystem
{
    public class ProgramDL
    {
        private static List<Program> programs = new List<Program>();

        public static bool AddProgram(Program program)
        {
            if (GetProgramById(program.ProgramId) == null)
            {
```

```
        programs.Add(program);
        return true;
    }
    return false;
}

public static List<Program> GetAllPrograms()
{
    return programs;
}

public static Program GetProgramById(string programId)
{
    foreach (var program in programs)
    {
        if (program.ProgramId.ToLower() == programId.ToLower())
            return program;
    }
    return null;
}

public static List<Program> GetAvailablePrograms()
{
    List<Program> available = new List<Program>();
    foreach (var program in programs)
    {
        if (program.HasAvailableSeats())
            available.Add(program);
    }
    return available;
}

public static bool UpdateProgram(Program program)
{
    for (int i = 0; i < programs.Count; i++)
    {
        if (programs[i].ProgramId.ToLower() == program.ProgramId.ToLo
wer())
        {
            programs[i] = program;
            return true;
        }
    }
    return false;
}

public static bool DeleteProgram(string programId)
{
    Program program = GetProgramById(programId);
    if (program != null)
    {
        programs.Remove(program);
        return true;
    }
    return false;
}
}
```

SubjectDL

```
using System;
using System.Collections.Generic;
namespace UniversityAdmissionSystem
{
    public class SubjectDL
    {
        private static List<Subject> subjects = new List<Subject>();

        public static bool AddSubject(Subject subject)
        {
            if (GetSubjectByCode(subject.SubjectCode) == null)
            {
                subjects.Add(subject);
                return true;
            }
            return false;
        }

        public static List<Subject> GetAllSubjects()
        {
            return subjects;
        }

        public static Subject GetSubjectByCode(string subjectCode)
        {
            foreach (var subject in subjects)
            {
                if (subject.SubjectCode.ToLower() == subjectCode.ToLower())
                    return subject;
            }
            return null;
        }

        public static List<Subject> GetSubjectsByCodes(List<string> codes)
        {
            List<Subject> result = new List<Subject>();
            foreach (var code in codes)
            {
                Subject subject = GetSubjectByCode(code);
                if (subject != null)
                    result.Add(subject);
            }
            return result;
        }

        public static bool UpdateSubject(Subject subject)
        {
            for (int i = 0; i < subjects.Count; i++)
            {
                if (subjects[i].SubjectCode.ToLower() == subject.SubjectCode.
ToLower())
                {
                    subjects[i] = subject;
                    return true;
                }
            }
            return false;
        }

        public static bool DeleteSubject(string subjectCode)
    }
}
```

```
        {
            Subject subject = GetSubjectByCode(subjectCode);
            if (subject != null)
            {
                subjects.Remove(subject);
                return true;
            }
            return false;
        }
    }
}
```

Main Program

```
namespace UniversityAdmissionSystem
{
```

```
    class UniversityApp
    {
        static void Main(string[] args)
        {
            InitializeSampleData();
            Menu.ShowMainMenu();
        }

        static void InitializeSampleData()
        {
            Program p1 = new Program("CS", "Computer Science", "Computing", 4
, 100);
            Program p2 = new Program("EE", "Electrical Engineering", "Enginee
ring", 4, 80);
            Program p3 = new Program("SE", "Software Engineering", "Computing
", 4, 60);
            Program p4 = new Program("AI", "Artificial Intelligence", "Comput
ing", 4, 50);

            ProgramDL.AddProgram(p1);
            ProgramDL.AddProgram(p2);
            ProgramDL.AddProgram(p3);
            ProgramDL.AddProgram(p4);

            Subject s1 = new Subject("CS101", "Programming Fundamentals", 4,
"Core", 5000);
            Subject s2 = new Subject("CS102", "Data Structures", 4, "Core", 5
500);
            Subject s3 = new Subject("MATH101", "Calculus", 3, "Core", 4000);
            Subject s4 = new Subject("PHY101", "Physics", 3, "Core", 4500);
            Subject s5 = new Subject("ENG101", "English", 2, "Core", 3000);
            Subject s6 = new Subject("CS201", "Database Systems", 3, "Core",
5000);
            Subject s7 = new Subject("CS202", "Algorithms", 4, "Core", 5500);
            Subject s8 = new Subject("AI301", "Machine Learning", 3, "Electiv
e", 6000);
            Subject s9 = new Subject("EE101", "Circuit Analysis", 4, "Core",
5000);

            SubjectDL.AddSubject(s1);
            SubjectDL.AddSubject(s2);
            SubjectDL.AddSubject(s3);
            SubjectDL.AddSubject(s4);
            SubjectDL.AddSubject(s5);
            SubjectDL.AddSubject(s6);
```

```
        SubjectDL.AddSubject(s7);
        SubjectDL.AddSubject(s8);
        SubjectDL.AddSubject(s9);

        p1.AddSubject(s1);
        p1.AddSubject(s2);
        p1.AddSubject(s3);
        p1.AddSubject(s4);
        p1.AddSubject(s5);
        p1.AddSubject(s6);
        p1.AddSubject(s7);

        p2.AddSubject(s4);
        p2.AddSubject(s3);
        p2.AddSubject(s5);
        p2.AddSubject(s9);

        p3.AddSubject(s1);
        p3.AddSubject(s2);
        p3.AddSubject(s3);
        p3.AddSubject(s5);
        p3.AddSubject(s6);

        p4.AddSubject(s1);
        p4.AddSubject(s2);
        p4.AddSubject(s3);
        p4.AddSubject(s8);
    }
}
```

Output:

```
=====
UNIVERSITY ADMISSION MANAGEMENT SYSTEM
=====

STUDENT MANAGEMENT:
  1. Add Student

PROGRAM MANAGEMENT:
  6. Add Program
  7. View All Programs
  8. Edit Program
  9. Delete Program
 10. Add Subject
 11. View All Subjects
 12. Edit Subject
 13. Delete Subject
 14. Add Subject to Program

ADMISSION PROCESS:
  0. Exit

Enter your choice: 1

===== ADD NEW STUDENT =====
  b. Back to Main Menu

Enter Student ID: 1
Enter Student Name: Zain
Enter Age (15-25): 19
Enter FSC Marks (0-1200): 939
Enter ECAT Marks (0-400): 221
Student added successfully!
```

Enter your choice: 2

```
===== ALL STUDENTS =====  
ID      | Name      | Age | FSC   | ECAT   | Status  
-----|-----  
1       | Zain      | 19  | 939.00 | 221.00 | Not Admitted  
=====
```

Press Enter to continue...

Enter Student ID to edit: 1

Editing Student: Zain (ID: 1)
Press Enter to keep current values.

Enter Name [Zain]:
Enter Age (15-25) [19]:
Enter FSC Marks (0-1200) [939]:
Enter ECAT Marks (0-400) [221]: 227
Student updated successfully!

Enter your choice: 4

```
===== DELETE STUDENT =====  
b. Back to Main Menu
```

Available Students:

```
ID      | Name      | Age | Status  
-----|-----  
1       | Zain      | 19  | Not Admitted
```

Enter Student ID to delete: 1

Are you sure you want to delete Zain (ID: 1)?
Confirm (y/n): n
Deletion cancelled.

Enter your choice: 5

===== ADD PROGRAM PREFERENCES =====

b. Back to Main Menu

Available Students:

ID	Name	Age	Merit Score
1	Zain	19	69.65

Enter Student ID: 1

Available Programs:

ID	Program Name	Department	Seats
CS	Computer Science	Computing	100
EE	Electrical Engineering	Engineering	80
SE	Software Engineering	Computing	60
AI	Artificial Intelligence	Computing	50

Enter program IDs separated by commas (e.g., P1,P2,P3):

EE

Added 1 program preferences.

Enter your choice: 6

===== ADD NEW PROGRAM =====

b. Back to Main Menu

Enter Program ID: CE

Enter Degree Title: Cmputer Engineering (NCEAC)

Enter Department: Electrical Department

Enter Duration (years): 4

Enter Total Seats: 3

Program added successfully!

Enter your choice: 7

===== ALL PROGRAMS =====

ID	Degree Title	Department	Duration	Total Seats	Available
CS	Computer Science	Computing	4	100	100
EE	Electrical Engineering	Engineering	4	80	80
SE	Software Engineering	Computing	4	60	60
AI	Artificial Intelligence	Computing	4	50	50
CE	Cmputer Engineering (NCEAC)	Electrical Department	4	3	3

Enter your choice: 8

===== EDIT PROGRAM =====

b. Back to Main Menu

Available Programs:

ID	Degree Title	Department	Seats
CS	Computer Science	Computing	100
EE	Electrical Engineering	Engineering	80
SE	Software Engineering	Computing	60
AI	Artificial Intelligence	Computing	50
CE	Computer Engineering (NCEAC)	Electrical Department	3

Enter Program ID to edit: CE

Editing Program: Computer Engineering (NCEAC) (ID: CE)
Press Enter to keep current values.

Enter Degree Title [Computer Engineering (NCEAC)]:
Enter Department [Electrical Department]: Engineering
Enter Duration (2-6) [4]:
Enter Total Seats [3]:
Program updated successfully!

Enter your choice: 9

===== DELETE PROGRAM =====

b. Back to Main Menu

Available Programs:

ID	Degree Title	Available Seats
CS	Computer Science	100
EE	Electrical Engineering	80
SE	Software Engineering	60
AI	Artificial Intelligence	50
CE	Computer Engineering (NCEAC)	3

Enter Program ID to delete: CS

Are you sure you want to delete Computer Science (ID: CS)?
Confirm (y/n): y
Program deleted successfully!

Enter your choice: 10

===== ADD NEW SUBJECT =====

b. Back to Main Menu

Enter Subject Code: CSC104

Enter Subject Name: OOP

Enter Credit Hours (1-6): 3

Enter Subject Type (Core/Elective): Core

Enter Fee Per Credit Hour: 3000

Subject added successfully!

Enter your choice: 11

===== ALL SUBJECTS =====

Code	Subject Name	Credit Hours	Type	Fee/CH
CS101	Programming Fundamentals	4	Core	5000.00
CS102	Data Structures	4	Core	5500.00
MATH101	Calculus	3	Core	4000.00
PHY101	Physics	3	Core	4500.00
ENG101	English	2	Core	3000.00
CS201	Database Systems	3	Core	5000.00
CS202	Algorithms	4	Core	5500.00
AI301	Machine Learning	3	Elective	6000.00
EE101	Circuit Analysis	4	Core	5000.00
CSC104	OOP	3	Core	3000.00

Enter your choice: 12

===== EDIT SUBJECT =====

b. Back to Main Menu

Available Subjects:

Code	Subject Name	Credit Hours	Fee/CH
CS101	Programming Fundamentals	4	5000.00
CS102	Data Structures	4	5500.00
MATH101	Calculus	3	4000.00
PHY101	Physics	3	4500.00
ENG101	English	2	3000.00
CS201	Database Systems	3	5000.00
CS202	Algorithms	4	5500.00
AI301	Machine Learning	3	6000.00
EE101	Circuit Analysis	4	5000.00
CSC104	OOP	3	3000.00

Enter Subject Code to edit: csc104

Editing Subject: OOP (Code: CSC104)

Press Enter to keep current values.

Enter Subject Name [OOP]:

Enter Credit Hours (1-6) [3]:

Enter Subject Type (Core/Elective) [Core]:

Enter Fee Per Credit Hour [3000]: 4000

Subject updated successfully!

Enter your choice: 13

===== DELETE SUBJECT =====

b. Back to Main Menu

Available Subjects:

Code	Subject Name	Credit Hours
-----	-----	-----
CS101	Programming Fundamentals	4
CS102	Data Structures	4
MATH101	Calculus	3
PHY101	Physics	3
ENG101	English	2
CS201	Database Systems	3
CS202	Algorithms	4
AI301	Machine Learning	3
EE101	Circuit Analysis	4
CSC104	OOP	3

Enter Subject Code to delete: cs101

Are you sure you want to delete Programming Fundamentals (Code: CS101)?

Confirm (y/n): y

Subject deleted successfully!

Enter your choice: 14

===== ADD SUBJECT TO PROGRAM =====

b. Back to Main Menu

Available Programs:

ID	Degree Title	Department	Seats
-----	-----	-----	-----
EE	Electrical Engineering	Engineering	80
SE	Software Engineering	Computing	60
AI	Artificial Intelligence	Computing	50
CE	Computer Engineering (NCEAC)	Engineering	3

Enter Program ID: ce

Available subjects:

CS102: Data Structures (4 CH)
MATH101: Calculus (3 CH)
PHY101: Physics (3 CH)
ENG101: English (2 CH)
CS201: Database Systems (3 CH)
CS202: Algorithms (4 CH)
AI301: Machine Learning (3 CH)
EE101: Circuit Analysis (4 CH)
CSC104: OOP (3 CH)

Enter Subject Code to add: cs201

Subject added to program successfully!

Enter your choice: 15

===== MERIT LIST =====

Rank	Student ID	Name	FSC	ECAT	Merit
1	1	Zain	939.00	227.00	69.65

=====

Enter your choice: 16

Processing admission based on merit and preferences...
Admission process completed. 1 students admitted.

Enter your choice: 17

===== ADMITTED STUDENTS =====

Student ID	Name	Program	Merit
1	Zain	Electrical Engineering	69.65

=====

Enter your choice: 18

===== PROGRAM ALLOCATION DETAILS =====

Student: Zain (ID: 1)
Merit Score: 69.65
Allocated Program: Electrical Engineering
Department: Engineering
Duration: 4 years

=====

```
Enter your choice: 19

===== SUBJECT REGISTRATION =====
  b. Back to Main Menu

Admitted Students:
ID      | Name      | Program      | Merit
-----|-----|-----|-----
1       | Zain      | Electrical Engineering | 69.65

Enter Student ID: 1

Available subjects for Electrical Engineering:
Code    | Subject Name      | Credit Hours | Type
-----|-----|-----|-----
PHY101  | Physics           | 3           | Core
MATH101 | Calculus          | 3           | Core
ENG101  | English           | 2           | Core
EE101   | Circuit Analysis  | 4           | Core

Max credit hours allowed: 9
Enter subject codes separated by commas (e.g., CS101,MATH101):
math101

Successfully registered 1 subjects (3 credit hours).

===== REGISTERED SUBJECTS FOR Zain =====
Code    | Subject Name      | Credit Hours | Type   | Fee
-----|-----|-----|-----|-----
MATH101 | Calculus          | 3           | Core   | 12000.00
Total Credit Hours: 3
Max Allowed: 9
=====
```

Explanation:

This is a University Admission Management System built as a console application in C# using a 3-Tier Layered Architecture (Presentation Layer, Business Logic Layer, Data Layer). The entry point in Program.cs seeds sample data — 4 programs (CS, EE, SE, AI) and 9 subjects — then launches the main menu loop. The Presentation Layer (PL\Menu.cs, ProgramView.cs, StudentView.cs, AdmissionView.cs) handles all user interaction: InputValidator provides reusable validation methods (GetValidString, GetValidInteger, GetValidDouble, GetYesNo,

and the newly added `GetValidChoice`) that loop until valid input is provided, and `Menu.ShowMainMenu()` renders a dynamic console menu that conditionally shows/hides options based on application state (e.g., "Register Subjects" only appears if admitted students exist). The Business Logic Layer contains entity models (`Student.cs`, `Program.cs`, `Subject.cs`) which encapsulate domain rules — Merit is calculated as $60\% \text{ FSC} + 40\% \text{ ECAT}$, programs enforce a 20-credit-hour cap, students are limited to 9 credit hours, and admission allocation iterates the merit-sorted list assigning each student to their highest preferred program with available seats. `MeritCalculator` sorts students by merit (ties broken by ECAT marks), `AdmissionManager` processes the allocation, `RegistrationManager` validates subject enrollment against program membership, duplicates, and credit limits, and `FeeCalculator` computes fees per subject ($\text{CreditHours} \times \text{FeePerCreditHour}$). The Data Layer (`StudentDL.cs`, `ProgramDL.cs`, `SubjectDL.cs`) uses in-memory `List<T>` collections with case-insensitive lookups and basic CRUD operations — all data is lost when the app exits. The flow is: add students and programs → assign program preferences to students → generate merit list → process admission (allocates seats by merit order) → register subjects for admitted students (up to 9 CH) → view fee breakdowns.

Conclusion:

In this lab, we learned how to design and implement a console-based University Admission Management System using 3-Tier Architecture. We understood the separation of concerns between the Presentation Layer (handling console menus, user input, and display), the Business Logic Layer (encapsulating domain rules like merit calculation, seat allocation, credit hour limits, and fee computation), and the Data Layer (managing in-memory storage with CRUD operations). We gained hands-on experience with input validation using reusable validator methods to prevent invalid data entry (e.g., restricting Subject Type to Core/Elective only, warning about invalid program IDs, and reporting unrecognized subject codes). We implemented key admission workflows: adding students and programs with preferences, generating a merit-sorted list ($60\% \text{ FSC} + 40\% \text{ ECAT}$), allocating seats to the highest-merit students, registering subjects within a 9-credit-hour cap, and calculating fees based on registered subjects. Through this project, we reinforced object-oriented principles, layered architecture patterns, and practical C# console application development.